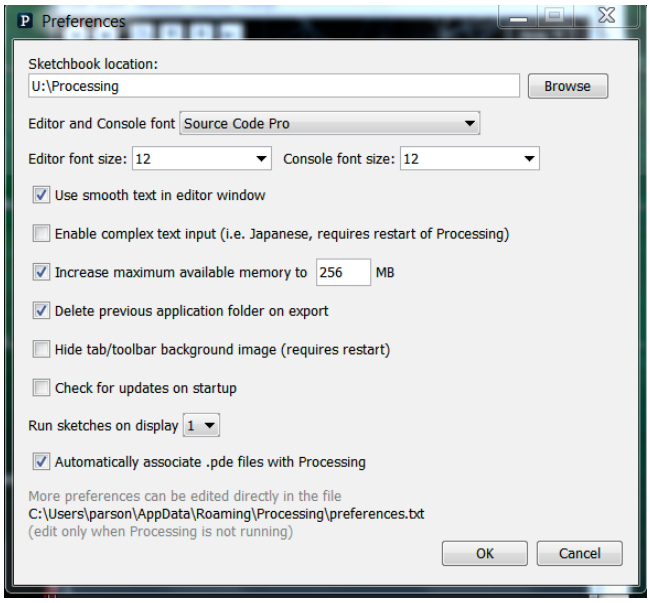
CSC 120 Introduction to Creative Graphical Coding, Spring 2021

Prof. Patrick Earl, Assignment 2, Implementing and testing a function that plots your avatar.
This assignment is due via **D2L link Assignment 2** by **11:59 PM on Friday March 19**.

When using Processing on the Kutztown campus Windows computers, make sure to start out **every time** by setting your Processing Preferences -> Sketchbook Location to U:\Processing. The U:\ drive is a networked drive that will save your work and make it accessible across campus. If you save it to your desktop or the lab PC you are using, you will lose your work when you log out. You must save it to the U:\ drive. If you do not have a folder called Processing under U:\, you must create one using the Windows Explorer. Processing Preferences is under the File menu on Windows.



If you have a working avatar from assignment 1, load it into Processing and **save it as sketch avatarFunction**. You will turn in file **avatarFunction.pde**. **The minimum size for your sketch should be 1500 by 750.**

If you do not have a working avatar, copy my stick figure avatarStick.pde from the Content section of D2L into a Processing window, save it as avatarFunction, and extend the number and type of shapes used in my stick figure to satisfy assignment 1. Add some real background and foreground scenery instead of the two shapes in avatarStick. If you do not make enhancements to these minimal background, foreground, and avatar figures, and satisfy assignment 1 requirements, you will lose points. **DO NOT USE avatarStick if your assignment 1 works. If you lost a few points in assignment 1, fix those problems in your avatarFunction**.

REQUIREMENTS:

1. Create a function avatar(…) that takes a minimum of three parameters, namely an X reference location, a Y reference location, and a float scale parameter similar to the avx, avy and avscale parameters in my avatar() example function. You may add other parameters for the wiggling body part, color changes, or other properties as you like.
2. Within function avatar(…), call **pushMatrix** as the first statement, **translate** as the second, **scale** as the third, and **popMatrix** as the last. Use the appropriate function parameters for translate and scale. Note that scale can take 2 parameters, one for X and another for Y; you could pass two scale parameters and use them in your function. The purpose of starting your function in this way is to position and scale your avatar differently without changing the avatar plotting code itself. The purpose of putting pushMatrix at the start is to record the coordinates in effect when your function is called from draw(), which is drawing the scenery; the call of popMatrix at the function's end is to restore those scenery coordinates.
3. Move all of your code from assignment 1 that actually draws the avatar into your avatar() function, after the above geometric transforms and *before* popMatrix, making any changes you find necessary to fit your avatar into this function. Change all global variable names within the function that have function parameter counterparts to the parameter names. Do not move scenery code into the function.[2]
4. Place a call to avatar(…) at the original location in the assignment 1 code, and ensure that the avatar still draws correctly.
5. Add some additional calls to avatar(…) within draw(), using different values for scale and the other parameters. You can plot multiple avatars at the same time, or avatars at different scales at different times.
6. Use an "if" construct at least once in your draw() or avatar() code to make a decision, for example to reverse direction. Using "else if" or default "else" portions is optional.
7. You must write at least one "while" or "for" loop that has some visible effect. My avatarFunction.pde use a "for" loop to plot multiple avatars at the same time. What you do with the repetition of a "while" or "for" loop is up to you.
8. All of the assignment 1 requirements for motion, moving body part, foreground & background scenery, etc. are still required. Assignment 2 adds to assignment 1 requirements.
9. Make sure to update the documentation at the top of your sketch. Add comments where useful to make the code clearer to understand.

Grading: Each of the above 9 steps is worth 10% each (90% total). Maintaining assignment 1 functionality earns 10% (or adding it if it was missing in assignment 1).

Drop avatarFunction.pde into D2L Assignment link labeled Assignment 02 by 11:59 PM on Friday March 19.

[2] Note that code to update global variables, such as my *avatarX*, *avatarXspeed*, *legX*, and *legXspeed* variables, must stay **outside** the avatar(...) function. Otherwise, every call to avatar(...) will update these variables. My code updates these variables immediately after the last call to avatar(...). There is no point in updating avatar(...)'s parameters or local variables just before the function ends, because parameters and local variables cease to exist after a function call completes.